Pion Electromagnetic Form Factor

Physics Note — PyQUDA Implementation

July 18, 2026

1 Physics Overview

The electromagnetic form factor (EMFF) of the pion, $F_\pi(Q^2)$, parametrizes the hadronic matrix element of the electromagnetic current between pion states. For a charged pion π^+, the matrix element is

$$\langle \pi^+(p_f)|J_\mu^{\rm em}(0)|\pi^+(p_i)\rangle = (p_i+p_f)_\mu\, F_\pi(Q^2), \qquad Q^2 = -q^2 = -(p_f-p_i)^2 \geq 0. \tag{1}$$

The electromagnetic current for a single quark flavor f with electric charge e_f is $J_\mu^{\rm em} = e_f\,\bar\psi\gamma_\mu\psi$. On the lattice this is accessed through the Euclidean three-point correlation function

$$C_3^\Gamma(\mathbf{q},\tau;\mathbf{p}_f,t_{\rm sep}) = \sum_{\mathbf{x}}\sum_{\mathbf{y}} e^{-i\mathbf{q}\cdot(\mathbf{x}-\mathbf{x}_0)} e^{-i\mathbf{p}_f\cdot(\mathbf{y}-\mathbf{x}_0)}\,\big\langle O_\pi(\mathbf{y},t_{\rm sep})\,J^\Gamma(\mathbf{x},\tau)\,O_\pi^\dagger(\mathbf{x}_0,0)\big\rangle, \tag{2}$$

where $O_\pi = \bar\psi\Gamma_{\rm sink}\psi$ is the pion interpolating operator, and $J^\Gamma = \bar\psi\Gamma\psi$ is the local current insertion. The initial pion momentum is fixed by momentum conservation: $\mathbf{p}_i = \mathbf{p}_f - \mathbf{q}$.

The pion interpolators project onto definite three-momentum via Fourier transforms at the sink (momentum $\mathbf{p}_f$) and the current insertion (momentum transfer $\mathbf{q}$). For the EMFF channel we are interested in the vector components of the current ($\Gamma = \gamma_\mu$ with $\mu = X, Y, Z, T$), but the implementation described here scans all 16 Dirac bilinears for diagnostic completeness and cross-checking.

After Wick contraction, the connected three-point function takes the trace form

$$C_3^\Gamma(\mathbf{q},\tau;\mathbf{p}_f,t_{\rm sep}) = \sum_{\mathbf{x}}\sum_{\mathbf{y}} e^{-i\mathbf{q}\cdot(\mathbf{x}-\mathbf{x}_0)} e^{-i\mathbf{p}_f\cdot(\mathbf{y}-\mathbf{x}_0)}\ \mathrm{tr}\Big[S_{\rm anti}(\mathbf{y},t_{\rm sep};\mathbf{x}_0,0)\,\Gamma_{\rm sink}\,S_q(\mathbf{y},t_{\rm sep};\mathbf{x},\tau)\,\Gamma\,S_q(\mathbf{x},\tau;\mathbf{x}_0,0)\,\Gamma_{\rm si} \tag{3}$$

Here $S_q(x,y)$ denotes the quark propagator from spacetime point y to x, and $S_{\rm anti}(x,y) = \gamma_5\,S_q(x,y)^\dagger\,\gamma_5$ is the antiquark propagator obtained via γ_5-hermiticity. The trace runs over spin and color indices. Equation (3) is the connected contribution; disconnected diagrams (where the current couples to a sea-quark loop) are not treated in this module.

2 Correlation Functions in Detail

2.1 Propagator Layout Convention

The PyQUDA `LatticePropagator` stores data in a nine-dimensional array with index layout

$$\texttt{prop.data} \sim (w,t,z,y,x,\alpha,\beta,c,d), \tag{4}$$

where

- $w \in \{0, 1\}$ is the checkerboard parity (even/odd site);
- (t, z, y, x) are spacetime coordinates (note the fine coordinate x is the last spatial index, reflecting the checkerboard decomposition);
- α is the spin index at the sink (final point);
- β is the spin index at the source (initial point);
- c is the color index at the sink;
- d is the color index at the source.

In prose indices throughout this note (and in the `einsum` patterns) we label these as $(w, t, z, y, x, \alpha, \beta, a, b)$ with α = spin_sink, β = spin_src, a = color_sink, b = color_src, or with equivalent index letters depending on context.

2.2 Three-Point Function

The connected three-point function, written with explicit spin and color indices, is

$$C_3^{\Gamma}(\mathbf{q}, \tau; \mathbf{p}_f, t_{\text{sep}}) = \sum_{\mathbf{x},\mathbf{y}} e^{-i\mathbf{q}\cdot(\mathbf{x}-\mathbf{x}_0)} e^{-i\mathbf{p}_f\cdot(\mathbf{y}-\mathbf{x}_0)} \times S_{\text{anti}}\big|_{\mathbf{y},t_{\text{sep}}}^{\alpha\beta;ab} \, (\Gamma_{\text{sink}})^{\beta\gamma} \, S_q\big|_{\mathbf{y},t_{\text{sep}};\,\mathbf{x},\tau}^{\gamma\delta;bc} \, (\Gamma)^{\delta\epsilon} \, S_q\big|_{\mathbf{x},\tau;\,\mathbf{x}_0,0}^{\epsilon\zeta;ca} \, (\Gamma_{\text{src}})^{\zeta\alpha}, \tag{5}$$

where repeated indices are summed over. The color trace closes because S_{anti} and S_q have opposite color flow directions, yielding $S_{\text{anti}ab} \, S_{q_{bc}} \, S_{q_{ca}}$ — a proper color singlet.

The intermediate antiquark-to-quark connection at the current insertion point $(\mathbf{x}, \tau)$ is given by the standard quark propagator $S_q(\mathbf{y}, t_{\text{sep}}; \mathbf{x}, \tau)$, representing the propagation from the current insertion up to the sink.

2.3 Two-Point Function

The pion two-point function, serving as a normalization and diagnostic, is

$$C_2^{\Gamma}(\mathbf{p}, t) = \sum_{\mathbf{x}} e^{-i\mathbf{p}\cdot(\mathbf{x}-\mathbf{x}_0)} \, \text{tr}\Big[S_{\text{anti}}(\mathbf{x}, t; \mathbf{x}_0, 0) \, \Gamma_{\text{sink}} \, S_q(\mathbf{x}, t; \mathbf{x}_0, 0) \, \Gamma_{\text{src}}\Big]. \tag{6}$$

For the standard pseudoscalar pion with $\Gamma_{\text{src}} = \Gamma_{\text{sink}} = \gamma_5$, this reduces to

$$C_2^{\gamma_5}(\mathbf{p}, t) = \sum_{\mathbf{x}} e^{-i\mathbf{p}\cdot(\mathbf{x}-\mathbf{x}_0)} \, \text{tr}\Big[\gamma_5 \, S_q(\mathbf{x}, \mathbf{x}_0)^\dagger \, \gamma_5 \, \gamma_5 \, S_q(\mathbf{x}, \mathbf{x}_0) \, \gamma_5\Big] = \sum_{\mathbf{x}} e^{-i\mathbf{p}\cdot(\mathbf{x}-\mathbf{x}_0)} \, \text{tr}\Big[S_q(\mathbf{x}, \mathbf{x}_0)^\dagger \, S_q(\mathbf{x}, \mathbf{x}_0)\Big]. \tag{7}$$

3 Sequential Source Method

The double sum over sink position $\mathbf{y}$ in Eq. (5) is computationally prohibitive if performed by nested loops. The standard solution is the *fixed-sink sequential source method*, which absorbs the sink sum into a single sequential inversion.

3.1 Construction of the Sequential Source

The sequential source is built on time slice $t_{\text{sep}} = t_{\text{src}} + t_{\text{insert}}$ (modulo L_t). Starting from the (anti)quark propagator $S_q^{\text{neg}}(\mathbf{y}, t_{\text{sep}}; \mathbf{x}_0, 0)$ with appropriate sink smearing applied, the source is

$$\eta_{\text{seq}}(\mathbf{y}; \mathbf{p}_f, t_{\text{sep}}) = \delta_{t_y,\, t_{\text{sep}}} \, e^{-i\mathbf{p}_f\cdot(\mathbf{y}-\mathbf{x}_0)} \, \Gamma_{\text{seq}} \, S_q^{\text{neg}}(\mathbf{y}, t_{\text{sep}}; \mathbf{x}_0, 0), \tag{8}$$

where the sequential gamma structure is

$$\Gamma_{\text{seq}} = \gamma_5 \, \Gamma_{\text{sink}}^{\dagger} \, \gamma_5. \tag{9}$$

The time-slicing operation $\delta_{t_y, t_{\text{sep}}}$ is handled by the utility function `sequential2`, which extracts the propagator data restricted to the specified time slice.

For smeared-smeared three-point functions the code applies the same sink-side smearing pattern to the active sequential line before the sequential inversion. If $\mathcal{S}_{w,\mathbf{k}}$ denotes the boosted Gaussian smearing operator defined in Sec. 7, the actual right-hand side used by the current production workflow is

$$\eta_{\text{seq}}^{SS}(\mathbf{y}; \mathbf{p}_f, t_{\text{sep}}) = \mathcal{S}_{w,\mathbf{k}_{\text{pos,sink}}} \left[\delta_{t_y, \, t_{\text{sep}}} \; e^{-i\mathbf{p}_f \cdot (\mathbf{y} - \mathbf{x}_0)} \; \Gamma_{\text{seq}} \; S_q^{\text{neg}, \, S \to S}(\mathbf{y}, t_{\text{sep}}; \mathbf{x}_0, 0) \right]. \tag{10}$$

Here $S_q^{\text{neg}, \, S \to S}$ already contains source smearing and the spectator-line sink smearing with $\mathbf{k}_{\text{neg,sink}}$, while the outer $\mathcal{S}_{w,\mathbf{k}_{\text{pos,sink}}}$ supplies the missing sink-side smearing for the active quark line represented by the sequential propagator. This is the meson analogue of the proton fixed-sink sequential source convention.

3.2 Sequential Inversion

The sequential propagator S_{seq} is obtained by solving the Dirac equation with η_{seq} as the source:

$$D(x, y) \; S_{\text{seq}}(y, x_0; \mathbf{p}_f, t_{\text{sep}}) = \eta_{\text{seq}}(x; \mathbf{p}_f, t_{\text{sep}}). \tag{11}$$

This inversion is performed using the same Dirac operator employed for the forward propagators. In the code, the call is `core.invertPropagator(dirac, src_seq, 1, 0)`.

3.3 Antiquark Conversion

After the sequential inversion, the resulting propagator is converted into an antiquark backward line using γ_5-hermiticity, exactly as is done for the two-point function:

$$S_{\text{seq}}^{\text{anti}}(x, x_0; \mathbf{p}_f, t_{\text{sep}}) = \gamma_5 \, S_{\text{seq}}(x, x_0; \mathbf{p}_f, t_{\text{sep}})^{\dagger} \, \gamma_5. \tag{12}$$

3.4 Sink-Summed Three-Point Function

With the sequential antiquark propagator in hand, the double sum over $\mathbf{y}$ has been absorbed into the sequential source construction. The three-point function reduces to a single spatial sum over the current insertion point $\mathbf{x}$:

$$\boxed{C_3^{\Gamma}(\mathbf{q}, \tau; \mathbf{p}_f, t_{\text{sep}}) = \sum_{\mathbf{x}} e^{-i\mathbf{q} \cdot (\mathbf{x} - \mathbf{x}_0)} \; \text{tr} \left[S_{\text{seq}}^{\text{anti}}(\mathbf{x}, \mathbf{x}_0; \mathbf{p}_f, t_{\text{sep}}) \, \Gamma \; S_q(\mathbf{x}, \tau; \mathbf{x}_0, 0) \, \Gamma_{\text{src}} \right].} \tag{13}$$

This is the expression evaluated by the `contract_EMFF` method in the code.

4 Two-Point Function (Implementation)

The two-point contraction implemented in `contract_2pt_pion` follows Eq. (6). After workflow-specific sink smearing, it calls the same shared pion C2 kernel used by pion EMT, qTMDWF, and qTMD. The procedure inside that kernel is:

1. Apply boosted smearing at the sink to both S_q (positive) and S_q^{neg} (negative) propagators.
2. Convert the negative propagator to an antiquark backward line: $S_{\text{anti}} = \gamma_5 (S_q^{\text{neg}})^{\dagger} \gamma_5$.
3. Insert the sink gamma on the backward line.

4. Contract the backward line, forward line, and source gamma into a spin-color trace.
5. Project onto definite momentum $\mathbf{p}$ using the Fourier phase.

The contribution from the sink-summed three-point signal is stored alongside the two-point correlators in the `HDF5` output tree under `data/c2pt/`.

5 Source Gamma Conventions

The source gamma matrix Γ_{src} in Eq. (3) can be specified through four modes, controlled by the parameter `src_gamma`:

Mode 1: `"fixed_g5"` — The source gamma is fixed to γ_5 regardless of the sink gamma. This is the standard choice for pseudoscalar pion measurements, where $\Gamma_{\text{src}} = \gamma_5$ projects onto the pion's γ_5 quantum numbers from the source.

Mode 2: `"same_as_sink"` — The source gamma is set equal to the sink gamma for each gamma channel separately: $\Gamma_{\text{src}} = \Gamma_{\text{sink}}$. This is useful for exploring the full Dirac structure of the correlation functions across all 16 bilinears.

Mode 3: `"dagger_of_sink"` — The source gamma is the γ_5-hermitian conjugate of the sink gamma: $\Gamma_{\text{src}} = \gamma_5 \, \Gamma_{\text{sink}}^{\dagger} \, \gamma_5$. This mode is motivated by the sequential source construction, where the same transformation appears.

Mode 4: **Any gamma label** — A specific gamma matrix label (e.g., `"5"`, `"T"`, `"X"`, etc.) can be provided directly, fixing Γ_{src} to that single gamma structure for all channels.

The implementation is in the function `_source_gamma_stack`, which takes as input the precomputed stack of 16 sink (or current) gamma matrices and returns the corresponding stack of source gamma matrices according to the chosen mode.

6 Gamma Matrix Convention

The code scans all 16 independent Dirac bilinears. The gamma matrices are labeled by the following mnemonics, ordered in the standard project convention:

Index	Label	Dirac Structure	QUDA Bitmask
0	5	γ_5	15
1	T	γ_4 (temporal)	8
2	T5	$\gamma_4\gamma_5$	7
3	X	γ_1 (spatial)	1
4	X5	$\gamma_1\gamma_5$	14
5	Y	γ_2 (spatial)	2
6	Y5	$\gamma_2\gamma_5$	13
7	Z	γ_3 (spatial)	4
8	Z5	$\gamma_3\gamma_5$	11
9	I	$\not{1}$ (identity)	0
10	SXT	σ_{xt}	9
11	SXY	σ_{xy}	3
12	SXZ	σ_{xz}	5
13	SYT	σ_{yt}	10
14	SYZ	σ_{yz}	6
15	SZT	σ_{zt}	12

The `QUDA` bitmask convention uses a 4-bit encoding where bit 0 corresponds to γ_x, bit 1 to γ_y, bit 2 to γ_z, and bit 3 to γ_t. Gamma products are formed by XOR of the constituent bit patterns. For the identity, the bitmask is 0; $\gamma_5 = \gamma_x\gamma_y\gamma_z\gamma_t$ corresponds to $1 + 2 + 4 + 8 = 15$. The tensor bilinears $\sigma_{\mu\nu} = \frac{1}{2}[\gamma_\mu, \gamma_\nu]$ correspond to the XOR of the individual bitmasks.

In the code, the gamma matrices are instantiated via `gamma.gamma`(n) where n is the `QUDA` bitmask index, and stored in a stacked array of shape $(16, 4, 4)$ by the helper function `_gamma_stack`.

7 Boosted Smearing

To improve the overlap of the interpolating operators with the pion ground state at non-zero momentum, the code employs *boosted Gaussian smearing* with momentum injection.

7.1 Kernel

The smearing kernel in position space is

$$K(\mathbf{r}; w, \mathbf{k}) = \exp\Big(-\frac{\mathbf{r}^2}{2w^2} \Big) \exp\left[2\pi i \left(\frac{k_x r_x}{L_x} + \frac{k_y r_y}{L_y} + \frac{k_z r_z}{L_z} \right) \right], \tag{14}$$

where w is the Gaussian width in lattice units and $\mathbf{k} = (k_x, k_y, k_z)$ is the integer momentum-smearing vector used in the code's `boost` argument. The coordinates $\mathbf{r}$ are centered periodic global lattice coordinates, e.g. $r_x \in [-L_x/2, L_x/2)$. The first factor is the Gaussian profile and the second factor injects the momentum $2\pi(k_x/L_x, k_y/L_y, k_z/L_z)$.

7.2 Implementation

The smearing is implemented in momentum space for efficiency. The source fermion is first Fourier-transformed (using `PyQUDA`'s distributed 3D FFT), multiplied by the Fourier transform of $K(\mathbf{r})$, and then inverse-transformed back to position space. This is done independently for each spin-color component of the propagator or fermion field.

Equivalently, for a fermion field ψ,

$$\psi_{\mathbf{k}}^{\text{smear}}(\mathbf{x}, t) = \mathcal{F}_{3d}^{-1} \left[\mathcal{F}_{3d}[\psi](\mathbf{p}, t)\, \mathcal{F}_{3d}[K_{\mathbf{k}}](\mathbf{p}) \right](\mathbf{x}, t). \tag{15}$$

The FFT is purely spatial; the same kernel is broadcast to every Euclidean time slice.

The smearing assumes the gauge links are in the identity gauge (or sufficiently smoothed), so no additional gauge rotation is applied.

7.3 Independent Boost Parameters

Pion measurements involve both a quark and an antiquark line, which may require different momentum injections. The code supports four independent boost vectors:

- `pos_boost_src`: momentum injected at the source for the forward (positive) quark propagator $S_q(x, x_0)$.
- `pos_boost_sink`: momentum injected at the sink for the forward quark propagator (relevant for the two-point function).
- `neg_boost_src`: momentum injected at the source for the backward (negative/antiquark) propagator.
- `neg_boost_sink`: momentum injected at the sink for the backward propagator; this is also applied before constructing the sequential source.

If the per-line source/sink boost parameters are not provided, they fall back to the global `pos_boost` and `neg_boost` values.

8 Code Implementation

The contraction module resides in `pyquda_measurement_utils/pion_EMFF_vibe_develop.py` and defines the class `pion_EMFF` with two principal methods. The sequential source is built by `create_meson_bw_seq_pyquda` in `pyquda_measurement_utils/bw_seq_pyquda.py`.

8.1 The Antiquark Backward Line: `_meson_backward_line`

Given a quark propagator $S(x, x_0)$, this function computes the antiquark line using γ_5-hermiticity:

$$S_{\text{anti}}(x, x_0) = \gamma_5 \, S(x, x_0)^\dagger \, \gamma_5. \tag{16}$$

The corresponding `einsum` pattern is

```
xp.einsum("ij, wtzyxilab, kl -> wtzyxkjba",
          gamma5, prop.data.conj(), gamma5)
```

In this pattern:

- $\texttt{gamma5}_{ij}$ (index i = row, j = column) acts from the left on the spin_sink index of the conjugated propagator.
- $\texttt{prop.data.conj()}_{\texttt{wtzyxilab}}$ has spin_sink i and spin_src l.
- $\texttt{gamma5}_{kl}$ (index k = row, l = column) acts from the right on the spin_src index.
- The output has spin_sink k, spin_src j, color_sink b, color_src a — the color indices are swapped relative to the input because conjugation exchanges sink and source.

8.2 Gamma Matrix Stack: `_gamma_stack`

This function constructs a stacked array of all 16 gamma matrices with shape $(16, 4, 4)$, labeled by the index g in subsequent contractions:

$$\Gamma_{g;\,ij} = \texttt{gamma\_ls}[g, i, j]. \tag{17}$$

8.3 Source Gamma Stack: `_source_gamma_stack`

Given the sink/current gamma stack and the `src_gamma` mode, this function returns the corresponding source gamma stack. For the four modes described in Section 5:

- `"fixed_g5"`: $\Gamma_{\text{src}} = \gamma_5$ for all g.
- `"same_as_sink"`: $\Gamma^{(g)}_{\text{src}} = \Gamma^{(g)}_{\text{sink}}$.
- `"dagger_of_sink"`: $\Gamma^{(g)}_{\text{src}} = \gamma_5(\Gamma^{(g)}_{\text{sink}})^\dagger\gamma_5$, implemented via `xp.einsum("ab, gbc, cd -> gad", gamma5, sink_gamma_ls.conj().swapaxes(1,2), gamma5)`.
- Specific label: $\Gamma^{(g)}_{\text{src}} = \Gamma_{\text{label}}$ for all g.

8.4 Two-Point Contraction: `contract_2pt_pion`

The contraction proceeds in three tensor steps:

Step 1 — Sink gamma insertion on the backward line:

```
bw_prop = xp.einsum("wtzyxjicf, gim -> gwtzyxjmcf",
                    bw_prop, sink_gamma_ls)
```

Here `bw_prop` $= S_{\text{anti}}(\mathbf{x}, t; \mathbf{x}_0, 0)$ with layout $(w, t, z, y, x, \alpha, \beta, c, d)$ where α = spin_sink, β = spin_src, c = color_sink, d = color_src. The sink gamma stack has layout (g, α, γ). The sum over α inserts the sink gamma between the backward line's spin_src and the forward line:

$$\big(S_{\text{anti}} \cdot \Gamma_{\text{sink}}\big)_{\alpha\gamma;cd} = S_{\text{anti}\,\alpha\beta;cd} \, (\Gamma_{\text{sink}})_{\beta\gamma}. \tag{18}$$

Step 2 — Spin-color trace:

```
corr_local = xp.einsum("gwtzyxjiab, wtzyxilba, glj -> gwtzyx",
                       bw_prop, prop_pos.data, source_gamma_ls)
```

The three tensors entering this contraction are:

- `bw_prop` (after Step 1): $(S_{\text{anti}} \cdot \Gamma_{\text{sink}})_{\alpha\beta;cd}$ with indices $(g, w, t, z, y, x, \alpha, \beta, c, d)$.
- `prop_pos.data`: $S_q(\mathbf{x}, t; \mathbf{x}_0, 0)$ with indices $(w, t, z, y, x, \beta, \gamma, d, c)$ — note that the color indices are transposed relative to the backward line.
- `source_gamma_ls`: $\Gamma^{(g)}_{\text{src}}$ with indices (g, γ, α).

The summed indices are:

- β — spin_src of backward line with spin_sink of forward line;
- γ — spin_src of forward line with spin_sink of source gamma;
- α — spin_sink of backward line with spin_src of source gamma (closing the spin trace);
- c — color_sink of backward line with color_src of forward line;
- d — color_src of backward line with color_sink of forward line (closing the color trace).

The output has shape (g, w, t, z, y, x) — the correlator for each gamma channel at each spacetime point.

In physics notation this is precisely:

$$\text{corr\_local}_g(\mathbf{x}, t) = \text{tr}_{\text{spin,color}}\Big[\big(S_{\text{anti}} \cdot \Gamma^{(g)}_{\text{sink}}\big)(\mathbf{x}, t)\; S_q(\mathbf{x}, t)\; \Gamma^{(g)}_{\text{src}}\Big]. \tag{19}$$

Step 3 — Momentum projection:

```
corr = xp.einsum("qwtzyx, gwtzyx -> gqt", phases, corr_local)
```

where $\texttt{phases}_{q,w,t,z,y,x} = e^{-i\mathbf{p}_q\cdot(\mathbf{x}-\mathbf{x}_0)}$ for each momentum mode q in `p_2pt`. The sum over spatial indices (z, y, x) and the checkerboard parity w is implicit in the `einsum` contraction of the spacetime axes.

8.5 Three-Point Contraction: `contract_EMFF`

The three-point contraction follows an identical spin-color trace structure, but replaces the backward line from the local propagator with the sequential antiquark propagator:

```
seq_bw_line = _meson_backward_line(seq_bw_prop)
current_inserted = xp.einsum(
    "wtzyxjicf, gim -> gwtzyxjmcf",
    seq_bw_line, current_gamma_ls)
corr_local = xp.einsum(
    "gwtzyxjiab, wtzyxilba, glj -> gwtzyx",
    current_inserted, prop_pos.data, source_gamma_ls)
corr = xp.einsum("qwtzyx, gwtzyx -> gqt", phases, corr_local)
```

The correspondence with the analytic expression Eq. (13) is:

- `seq_bw_line` is $S^{\text{anti}}_{\text{seq}}(x, x_0; \mathbf{p}_f, t_{\text{sep}})$ from Eq. (12).
- `current_gamma_ls` is the stack of 16 gamma matrices for the current insertion Γ.
- The `einsum wtzyxjicf, gim -> gwtzyxjmcf` inserts the current gamma Γ into the sequential backward line, producing $\big(S^{\text{anti}}_{\text{seq}} \cdot \Gamma\big)$.
- The second `einsum` performs the full spin-color trace $\text{tr}\big[S^{\text{anti}}_{\text{seq}} \cdot \Gamma \cdot S_q \cdot \Gamma_{\text{src}}\big]$.
- The final `einsum` projects onto momentum transfer $\mathbf{q}$ via Fourier phase $e^{-i\mathbf{q}\cdot(\mathbf{x}-\mathbf{x}_0)}$.

Result layout: After gathering across MPI ranks, the output array has shape $(\texttt{nq}, \texttt{ngamma}, L_t)$ or equivalently (g, q, t). In the main application code, the axes are transposed to $(\texttt{ngamma}, \texttt{nq}, t_{\text{sep}}+2)$ for storage.

8.6 Sequential Source Construction

The sequential source is built by `create_meson_bw_seq_pyquda` in `bw_seq_pyquda.py`. The steps are:

1. **Time slicing:** The input propagator $S^{\text{neg}}(y, x_0)$ is restricted to time slice t_{sink} using `sequential12`(`prop`, t_{sink}).
2. **Momentum phase:** Multiply by $e^{-i\mathbf{p}_f\cdot(\mathbf{y}-\mathbf{x}_0)}$ on the spatial lattice sites at the sink time slice.

```
seq_data = xp.einsum("wtzyx, wtzyxilab -> wtzyxilab",
                     mom_phase, seq_data)
```

3. **Gamma insertion:** Apply $\Gamma_{\text{seq}} = \gamma_5\Gamma^\dagger_{\text{sink}}\gamma_5$.

```
gamma_seq = xp.einsum("ab, cb, cd -> ad",
                      G5, sink_gamma.conj(), G5)
seq_data = xp.einsum("ij, wtzyxjlab -> wtzyxilab",
                     gamma_seq, seq_data)
```

4. **Boosted smearing:** The sequential source is smeared with the same width and boost as the original propagator, `boosted_smearing`(`src`, w, `boost`).
5. **Inversion:** Solve $D\,S_{\text{seq}} = \eta_{\text{seq}}$ via `core.invertPropagator`(`dirac`, `src_seq`, $1, 0$).
6. **Final transformation:** Apply γ_5 from the right to properly close the spin structure: $S_{\text{seq}} \to S_{\text{seq}}\,\gamma_5$. (This is a convention for the subsequent antiquark conversion in `contract_EMFF`.)

```
final_term = xp.einsum("wtzyxijfc, ik -> wtzyxjkcf",
                       prop_smeared.data.conj(), G5)
```

The complete sequential source is thus:

$$\eta_{\text{seq}}(y; \mathbf{p}_f, t_{\text{sep}}) = \delta_{t_y,\,t_{\text{sep}}}\; e^{-i\mathbf{p}_f\cdot(\mathbf{y}-\mathbf{x}_0)}\; \left(\gamma_5\,\Gamma^\dagger_{\text{sink}}\,\gamma_5\right)\; S^{\text{neg}}(y, x_0). \tag{20}$$

9 HDF5 Output Structure

The two-point and three-point correlators are saved in `HDF5` format under the data directory specified by the user (typically `./data/`).

9.1 Two-Point Data

Path template:

```
data/c2pt/<lat>.c2pt.<cfg>.<ama>.<src_pos>.<sm_tag>.src<SRC>.h5
```

Internal structure:

```
SS/
  "5"/
    PX0PY0PZ0    : array(shape=(Lt,), dtype=complex)
    PX0PY0PZ1    : ...
    ...
  "T"/
    ...
  ...
```

Under the SS group, each gamma label contains datasets labeled by momentum PX<px>PY<py>PZ<pz>. Each dataset is a 1D complex array over time slices, already time-rolled so that the source is at $t = 0$. The two-point function is saved via save_proton_c2pt_hdf5, which also parses the source time from the tag to perform the roll. The root attributes record src_interpolator=<SRC> and sink_interpolator=all_16_gamma_scan.

9.2 Three-Point (EMFF) Data

Path template:

```
data/pion_EMFF/<lat>.pion_EMFF.<cfg>.<ama>.<src_pos>.
    <sm_tag>.src<SRC>.sink<SINK>.<pf_tag>.h5
```

where <pf_tag> = PX<px>PY<py>PZ<pz>dt<t_insert> The file contains all 16 current gamma channels. Its root attributes record src_interpolator, sink_interpolator, and current_gamma_basis=all_1 For a multi-source-interpolator run, the complete sorted source list and the sink interpolator enter the sample-log identity. The <sm_tag> itself describes only preprocessing, smearing, and boost parameters.

Lightweight resume. Each t_{sep} has a text sample log. Rank 0 reads all logs once and broadcasts their exact completed-source sets. If every requested t_{sep} is complete, the source is skipped before inversion; otherwise only the missing separations are contracted and written. A completion line is durably appended only after every requested fixed-source-Gamma file for that source and separation has closed. The log is the sole resume state and does not probe HDF5 files that may already have been transferred. The production interface names the requested list t_separations; for example, -t_separations 2,3,4. Each loop iteration passes one scalar t_{sep} to sequential-source construction. The current log name assumes fixed run flags, $q_{\max}$, momenta, source Gamma list, and separation list. A changed physical grid must use a new data/setup identity, and concurrent writers to one log are unsupported.

Internal structure:

```
SS/
  "5"/
    PX0PY0PZ0    : array(shape=(tsep+2,), dtype=complex)
    PX1PY0PZ0    : ...
    ...
```

Under the SS group, the single gamma label (the file contains one gamma channel per file after MPI-distributed writing) contains datasets labeled by momentum transfer PX<qx>PY<qy>PZ<qz>. Each dataset is a 1D complex array of length $t_{\text{sep}} + 2$, indexed by the current insertion time slice τ.

The saving is performed by save_pion_EMFF_hdf5_noRoll, which does not apply any time roll (the source-time alignment is handled during post-processing).

9.3 File Naming Convention Summary

Component	Description
<lat>	Lattice tag (e.g., S8T32)
<cfg>	Configuration number
<ama>	AMA label (e.g., EMFF.ex)
<src_pos>	Source position x<x>y<y>z<z>t<t>
<sm_tag>	Smearing tag (HYP steps, width, boost vectors)
<SRC>	Source interpolator label
<SINK>	Sink interpolator label
<pf_tag>	Final momentum and time separation

The tag-building helpers are defined in `pyquda_measurement_utils/io_corr.py`:

- `get_c2pt_file_tag` — builds the two-point HDF5 path.
- `get_pion_EMFF_file_tag` — builds the three-point HDF5 path.
- `save_proton_c2pt_hdf5` — writes two-point data (shared with baryon workflows).
- `save_pion_EMFF_hdf5_noRoll` — writes three-point EMFF data.

10 Workflow Summary

The complete pion EMFF measurement, as orchestrated by the main application script `Pyquda_pion_EMFF.py`, proceeds as follows:

1. **Gauge field preparation:** Read the gauge configuration, apply HYP smearing (1 step, $\alpha = (0.75, 0.6, 0.3)$), set antiperiodic temporal boundary conditions.
2. **Dirac operator:** Construct a Wilson-clover Dirac operator with the specified mass, c_{SW}, and multigrid parameters.
3. **Source creation:** Generate point sources at distributed spacetime positions.
4. **Positive quark propagator:** Apply boosted smearing at the source (`pos_boost_src`), invert.
5. **Negative quark propagator:** Apply boosted smearing at the source (`neg_boost_src`) if different from positive; otherwise copy the positive propagator.
6. **Two-point contraction:** Apply sink smearing to both propagators, contract via `contract_2pt_pion`, save to `data/c2pt/`.
7. **Sequential propagator:** Apply sink smearing to the negative propagator (`neg_boost_sink`), build the sequential source at t_{sink}, invert, and post-process.
8. **Three-point contraction:** Contract the sequential propagator with the positive forward propagator and all 16 current gamma structures, project onto momentum transfer, save per-gamma-channel files to `data/pion_EMFF/`.

11 Code Source Reference

The primary source files comprising the pion EMFF measurement are:

- `pyquda_measurement_utils/pion_EMFF_vibe_develop.py` — Defines the `pion_EMFF` class with `contract_2pt_pion` and `contract_EMFF`, the gamma matrix stacks, the antiquark backward line conversion, and the source gamma mode logic.
- `pyquda_measurement_utils/bw_seq_pyquda.py` — Defines `create_meson_bw_seq_pyquda` for building the fixed-sink meson sequential source, time slicing, and inversion. Also contains `create_bw_seq_pyquda` for baryon sequential sources (used in other workflows).
- `pyquda_measurement_utils/boosted_smearing_pyquda.py` — Implements the FFT boosted Gaussian smearing kernel with momentum injection via distributed 3D FFT.

- `pyquda_measurement_utils/io_corr.py` — HDF5 I/O helpers for two-point, three-point, and other correlator output formats.
- `pyquda_measurement_utils/tools.py` — Backend-agnostic utilities (`_get_xp_from_array`, `_asarray_on_queue`, `mpi_print`).
- `application/EMFF_pion/perlmutter/Pyquda_pion_EMFF.py` — Main application script that orchestrates the full measurement workflow, handling gauge I/O, propagator generation, and HDF5 output.

The sequential source time-slicing utility `sequential12` is provided by `pyquda_utils.source`. The momentum phase factors are generated by `pyquda_utils.phase.MomentumPhase`.